

nanochat

Full ChatGPT-Style Pipeline From Scratch

GPT-2 Architecture · Pretraining · SFT · REINFORCE · KV-Cache · Apple M2 Pro (MPS) · 36/36 tests passing

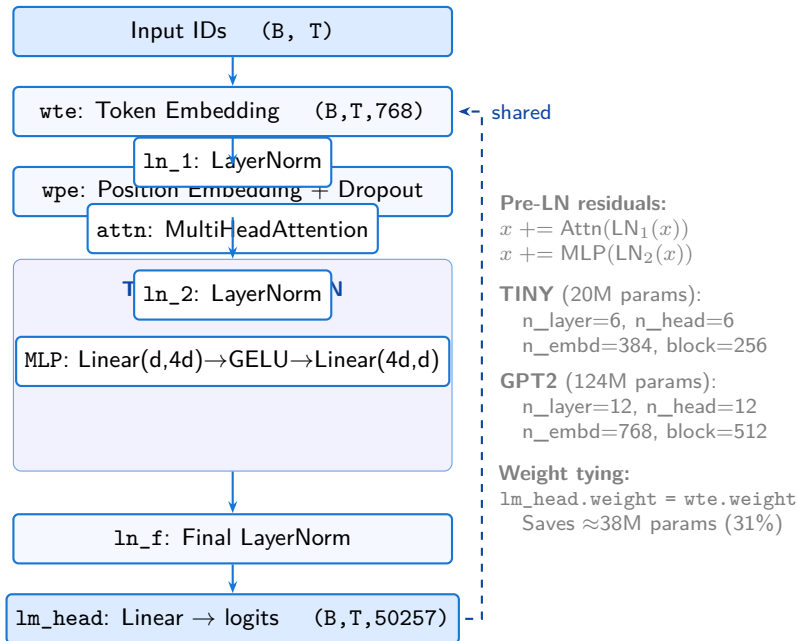
P#	PStage	PWhat it does	PFile(s)	PKey metric
1	Architecture	Build GPT-2 transformer from scratch	model/	124M or 20M params
2	Pretraining	Predict next token on raw text	pretrain/	Loss: 10.83
3	SFT	Fine-tune on conversations; mask non-assistant tokens	sft/	Matches reference
4	RL	REINFORCE + KL penalty	rl/	500 steps, G=8
5	Inference	KV-cache: $O(T)$ vs naive $O(T^2)$	inference/	2.1x speedup
6	Benchmarks	Loss curves, throughput, eval	benchmarks/	114 vs 239 tok/s
7	Notebooks	7 runnable notebooks, top-to-bottom	notebooks/	All execute OK

Contents

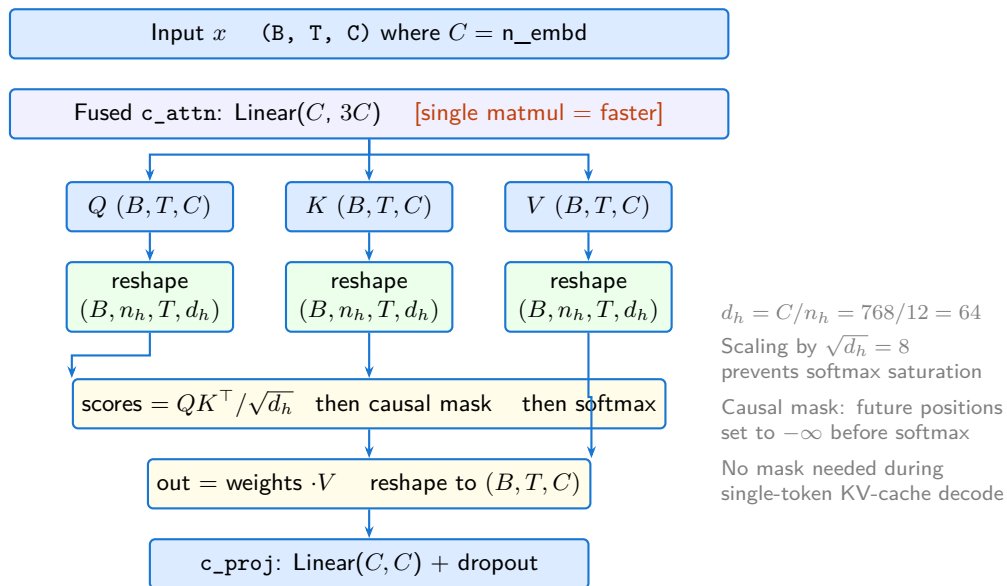
1	Transformer Architecture	3
1.1	Full Model Diagram	3
1.2	Multi-Head Attention Diagram	3
1.3	Architecture Comparison Table	3
2	GPT-2 Weight Loading	4
3	Pretraining	4
3.1	Data Pipeline	4
3.2	Training Setup	4
3.3	Why the Initial Loss Equals $\ln(50257)$	4
4	Supervised Fine-Tuning (SFT)	4
4.1	ChatML Format and What Gets Masked	4
4.2	SFT Loss Function	4
4.3	The Critical Denominator Choice	5
5	Reinforcement Learning	5
5.1	RL Loop Diagram	5
5.2	Key RL Equations and Intuition	5
6	KV-Cache Inference	5
6.1	Naive vs Cached: Visual	5
6.2	Complexity Comparison	5
7	All 6 Challenges	6
8	Interview Questions — Standard Level	6
9	Bonus Questions — Frontier Lab Depth	6
10	Quick Reference	7

1 Transformer Architecture

1.1 Full Model Diagram



1.2 Multi-Head Attention Diagram



1.3 Architecture Comparison Table

PDesign choice	PWhat we use	PWhy
Normalization	Pre-LN ($x += f(\text{LN}(x))$)	Stable gradients; no warmup needed
Attention	Fused QKV (one Linear, 3× output)	One matmul is faster than three; cleaner code
Causal mask	<code>register_buffer</code> (lower triangular)	Moves to device with model; not a parameter
MLP width	4× expansion ($d \rightarrow 4d \rightarrow d$)	Empirically optimal for GPT-class models
MLP activation	GELU	Smoother than ReLU; GPT-2 standard
Weight tying	<code>lm_head.weight = wte.weight</code>	Saves 38M params; principled (same role)
Init std	0.02 (Normal)	Keeps residual stream small at init

2 GPT-2 Weight Loading

PLayer	PHF Conv1D shape	POur Linear shape	PFix required
c_attn.weight	$(C, 3C) = (\text{in}, \text{out})$	$(3C, C) = (\text{out}, \text{in})$	.t() transpose
c_proj.weight	(C, C) transposed	(C, C) normal	.t()
mlp.c_fc.weight	$(C, 4C)$	$(4C, C)$	.t()
mlp.c_proj.weight	$(4C, C)$	$(C, 4C)$	.t()
All others	correct	correct	direct copy_()

Listing 1: Weight loader transpose check (model/gpt.py)

```

1 needs_transpose = any(hf_key.endswith(t) for t in
2   ["attn.c_attn.weight", "attn.c_proj.weight", "mlp.c_fc.weight", "mlp.c_proj.weight"])
3 with torch.no_grad():
4     if needs_transpose:
5         sd[our_key].copy_(hf_val.t())    # .t() flips (in,out) -> (out,in)
6     else:
7         sd[our_key].copy_(hf_val)

```

3 Pretraining

3.1 Data Pipeline



3.2 Training Setup

PHyperparameter	PValue	PWhy
LR schedule	Cosine	High LR early (fast), decays smoothly to near-zero (precise)
Warmup steps	200	Random init: high LR too early can diverge before weights stabilize
Grad clip	1.0	Bounds gradient norm; one bad batch cannot corrupt weights
Init loss	10.82	$\ln(50257)$: sanity check that weights are uniform over vocab
Measured init	10.8294	Confirms correct data pipeline and no bugs in loss computation
Device	MPS	Metal Performance Shaders – Apple Silicon GPU

3.3 Why the Initial Loss Equals $\ln(50257)$

At initialization all weights $\sim \mathcal{N}(0, 0.02)$, so logits ≈ 0 . Softmax of all-zero logits is uniform over 50257 tokens. Cross-entropy of a uniform distribution: $-\log(1/50257) = \ln(50257) \approx 10.82$. Our measured value is 10.8294 – gap < 0.01 confirms correct pipeline.

4 Supervised Fine-Tuning (SFT)

4.1 ChatML Format and What Gets Masked

4.2 SFT

Loss

Function

Listing 2: Complete SFT loss (sft/trainer.py)

```

1 def sft_loss(model, input_ids, loss_mask):
2     B, T = input_ids.shape
3     logits = model(input_ids)                # (B, T, vocab_size)
4     logits = logits[:, :-1, :].contiguous()  # drop last (no target)
5     targets = input_ids[:, 1:].contiguous()  # shift: target[t] = input[t+1]
6     mask = loss_mask[:, 1:].contiguous()     # align mask with shifted targets
7
8     per_token = F.cross_entropy(logits.view(-1, vocab_size),
9                                targets.view(-1), reduction="none").view(B, T-1)
10
11     denom = mask.sum()
12     if denom == 0:
13         return torch.tensor(0.0, requires_grad=True)
14     return (per_token * mask).sum() / denom  # average over ASSISTANT tokens only

```

4.3 The Critical Denominator Choice

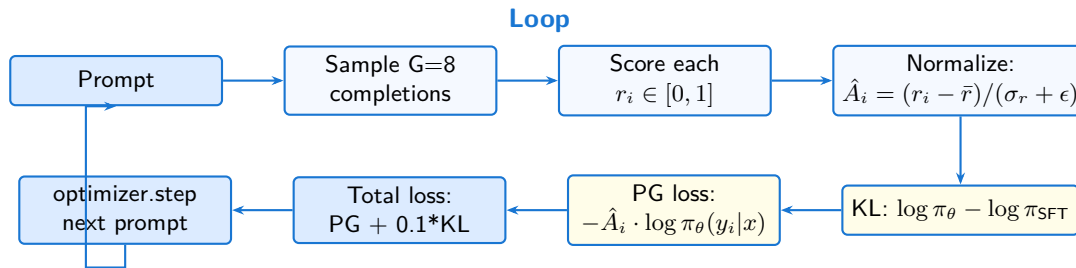
PScenario	Pmask.sum()	Pseq_len	PEffect if using seq_len
Short prompt + long answer (10 + 100 tokens)	100	110	1.1x dilution — acceptable
Long prompt + short answer (500 + 10 tokens)	10	510	51x dilution — gradient disappears

5 Reinforcement

Learning

5.1 RL

Diagram



5.2 Key

RL

Equations

and

Intuition

PFormula	PIntuition
$\hat{A}_i = (r_i - \bar{r}) / (\sigma_r + 10^{-8})$	Group-relative: "this was 2 std above group mean." Scale-invariant. ϵ prevents div-by-zero when all rewards are identical.
$\mathcal{L}_{PG} = -\hat{A}_i \cdot \log \pi_\theta(y_i x)$	Negative sign: PyTorch minimizes, we want to maximize reward. High- $\hat{A}$ completions get increased log-prob.
$\mathcal{L}_{KL} = \log \pi_\theta(y) - \log \pi_{SFT}(y)$	Divergence from frozen SFT ref. Prevents reward hacking (gaming rules with degenerate text). KL coeff 0.1 = 1 nat of KL costs 0.1 loss.
$\mathcal{L} = \mathcal{L}_{PG} + 0.1 \cdot \mathcal{L}_{KL}$	Final loss. Tradeoff: $\alpha \rightarrow 0$ = pure RL, hacking. $\alpha \rightarrow \infty$ = frozen model, no learning.

6 KV-Cache

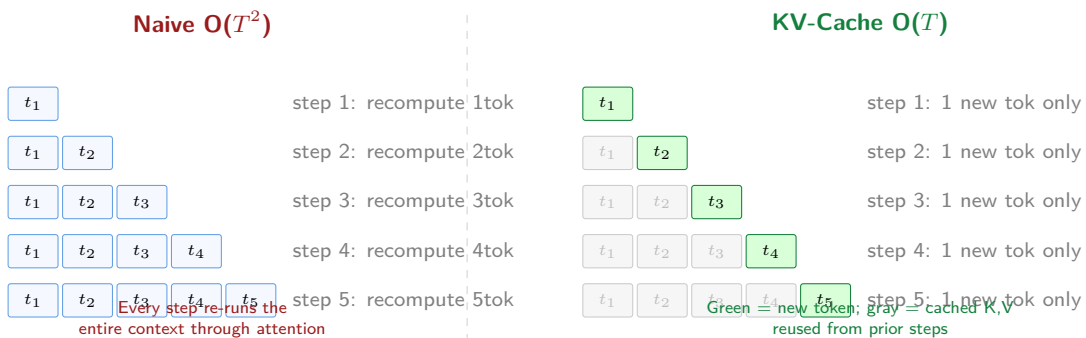
Inference

6.1 Naive

vs

Cached:

Visual



6.2 Complexity

Comparison

PProperty	PNaive	PKV-Cache	PNotes
Time per step	$O(T^2)$	$O(T)$	T = total tokens so far
Memory usage	$O(1)$	$O(T \cdot L \cdot H \cdot d_h)$	Grows with context length
Throughput (M2 Pro)	114.3 tok/s	239.1 tok/s	Measured on MPS
Speedup	1.0x	2.1x	Increases with longer sequences
KV size (512 ctx, GPT2)	—	≈37 MB	$2 \times 12 \times 12 \times 512 \times 64 \times 4$ bytes

Listing 3: Prefill + decode with pos_offset (inference/generate.py)

```

1 logits_prefill = model(ids, pos_offset=0) # run full prompt ONCE
2 next_tok = _sample_token(logits_prefill[0, -1], ...) # first NEW token from prefill
3 generated, last_tok = [next_tok.item()], next_tok.unsqueeze(0)
4
5 for step in range(1, max_new):
6     pos = T_prompt + step - 1 # absolute position -- critical for parity
7     logits = model(last_tok, pos_offset=pos)
8     next_tok = _sample_token(logits[0, -1], ...)
9     generated.append(next_tok.item())
10    last_tok = next_tok.unsqueeze(0)

```

7 All

6

Challenges

P#	PBug	PRoot cause	PFix	PKey lesson
1	SFT loss off-by-one	logit[t] predicts token[t+1]; shift must be applied to logits, targets, AND mask	logits[:, :-1], targets[:, 1:], mask[:, 1:]	Draw index alignment before coding
2	Wrong denominator	seq_len dilutes gradient 51x on long prompts + short answers	(loss*mask).sum() / mask.sum()	Denominator controls effective LR
3	Conv1D transpose	HF stores (in, out); nn.Linear expects (out, in); 4 layers affected	.t() on those 4 weight tensors	Validate logits vs HF reference
4	MPS JIT warmup	First MPS call compiles graph: 10x slower than subsequent calls	One dummy forward before timing loop	Report mean of warm runs, not first
5	RL NaN loss	All G=8 completions get same reward; std=0; division produces NaN	rewards.std() + 1e-8	Assert not isnan(loss) in RL
6	KV-cache off-by-one (2 bugs)	Wrong first token + positional embeddings start at 0 not $T_{\text{prompt}} + \text{step}$	Sample from prefill logits + pos_offset patch	Multi-seed parity test

8 Interview

Questions

Standard

Level

Dot products have $\text{std} = \sqrt{d_k}$. With $d_k = 64$, $\text{std} = 8$: softmax is nearly one-hot, gradients vanish. Dividing by $\sqrt{64} = 8$ restores $\text{std} \approx 1$, keeps all heads trainable.

`lm_head.weight = wte.weight` shares the same tensor. Same job, opposite directions. Saves $50257 \times 768 \approx 38.6\text{M}$ params (31% of GPT-2). Test with `is`, not `==`.

Only assistant tokens contribute to numerator. Using `seq_len`: 500-token prompt + 10-token answer = 51x dilution. `mask.sum() = 10` = correct denominator. Loss is always average over assistant tokens only.

Keys+values for prior tokens don't change between steps. Compute once (prefill), cache, reuse. Each new step processes ONE token: $O(T)$ not $O(T^2)$. Tradeoff: $O(T \cdot L \cdot H \cdot d_h)$ memory grows with context.

Normalize within G completions: $\hat{A}_i = (r_i - \bar{r}) / (\sigma_r + \epsilon)$. "Reward 0.7 when mean=0.3, std=0.2" becomes $+2.0\sigma$ signal. Scale-invariant across prompts. $\epsilon = 10^{-8}$ prevents div-by-zero.

Without it: reward hacking. Model finds degenerate completions that score high but are nonsense (e.g., "!!!!!!" for punctuation reward). KL anchors to frozen SFT ref. $\alpha = 0.1$: 1 nat of KL costs 0.1 in loss.

Pre-LN: $x += f(\text{LN}(x))$. Residual bypasses LN – clean gradient highway. Post-LN puts LN on residual, causing gradient instability. Pre-LN: higher LR safe, no warmup required. GPT-2 onward uses Pre-LN.

KV-cache off-by-one. Passed seed=42 by coincidence (all 8 tokens were identical value). Failed 9/10 other seeds. Two bugs: wrong first token + positions started at 0. Found by: `torch.equal(naive, cached)` for seeds 0,1,2.

9 Bonus

Questions

Frontier

Lab

Depth

$q, k \sim \mathcal{N}(0, 1)$. $q \cdot k = \sum q_i k_i$. Each term variance $= E[q_i^2 k_i^2] = 1$. Sum of d_k iid: $q \cdot k \sim \mathcal{N}(0, d_k)$, std $= \sqrt{d_k}$. Dividing by $\sqrt{d_k}$ normalizes to std=1.

This: REINFORCE + rule-based reward. RLHF: PPO + learned reward model from human A-vs-B comparisons. PPO clips update ratio (stable). Reward model from thousands of labels. KL structure identical. This is RLHF-lite: same architecture, no human labels.

$2 \times L \times B \times H \times T \times d_h \times \text{bytes}$. Hypothetical GPT-4 ($L=96, H=96, T=128K, d_h=128, \text{bf16}$): ≈ 305 GB. Solutions: GQA (share KV heads), sliding-window eviction, int8 KV quant, PagedAttention.

$\mathcal{L} = -\mathbb{E}[r] + \alpha D_{\text{KL}}$. $\alpha \rightarrow 0$: pure RL, degenerate outputs. $\alpha \rightarrow \infty$: frozen at SFT. Rule: larger α when reward is noisy; smaller when trustworthy. Sweet spot: 0.1–0.3 for demos.

Prefill: need lower-triangular mask ($T \times T$). Decode: query is LAST position – should attend to ALL cached tokens – no mask needed. Applying prefill mask to a decode step clips incorrectly. Fix: skip mask when $T = 1$.

Architecture: GQA, RoPE, bf16. Training: FSDP (model sharding), activation checkpointing (recompute in backward), grad clip critical. Inference: Flash Attention ($O(T)$ memory via tiling), speculative decoding (draft + verify), PagedAttention for batching.

Post-LN backward: $\partial \mathcal{L} / \partial x$ passes through $\partial \text{LN} / \partial (x + f(x))$. At depth, $x + f(x)$ has large norm; LN Jacobian $\propto 1/\text{norm}$ destabilizes. Pre-LN: $\partial \mathcal{L} / \partial x = I + (\dots)$. I is gradient highway – exists regardless of depth. LN input always unit variance.

Weights $\sim \mathcal{N}(0, 0.02)$: logits ≈ 0 . Softmax of zeros = uniform over 50257. Cross-entropy of uniform: $-\log(1/50257) = \ln(50257) \approx 10.82$. Measured: 10.8294. Gap < 0.01 = no pipeline bugs.

10 Quick

Reference

Key Equations

PName	PFormula
Attention	$\text{softmax}(QK^\top / \sqrt{d_k})V$
SFT loss	$\sum_t \ell_t m_t / \sum_t m_t$
Advantage	$(r_i - \bar{r}) / (\sigma_r + \epsilon)$
RL total loss	$\mathcal{L}_{\text{PG}} + 0.1 \cdot \mathcal{L}_{\text{KL}}$
Cosine LR	$\text{lr_min} + \frac{1}{2} \Delta (1 + \cos \frac{\pi t}{T})$

Function Signatures

Signature and return type

`GPT.forward(idx[B,T])` → [B,T,V]

`get_batch(split, block, batch, dev)` → (x,y)

`build_batch(convos, enc, block)` → (ids,mask)|None

`sft_loss(model, ids, mask)` → scalar

`compute_reward(text:str)` → float ∈ [0,1]

`generate_naive(model, ids, n)` → tokens

`generate_cached(model, ids, n)` → tokens (2.1x)